

Stratégie de Tests

Intégration et tests des composants logiciels

Ce rapport présente la stratégie de tests de CareerAI : pyramide de tests, cas couverts en V1, indicateurs de couverture, mécanismes de non-régression et roadmap d'extension vers les tests d'intégration et E2E.

Projet	CareerAI — Plateforme SaaS de recherche d'emploi
Stack	Next.js 14 · Supabase · Groq IA · Resend
URL	https://www.careerai.live
GitHub	github.com/ARTHURNJOUONANG/Projet_Chat_board

1. État actuel et contexte

■ Le projet dispose historiquement de tests manuels (scripts/test-saas-api.js). La V1 de tests automatisés ajoute une base pragmatique avec Vitest pour les tests unitaires et un script de smoke pour les endpoints critiques.

```
npm test # Tests unitaires Vitest
npm run test:coverage # Couverture sur périmètre testé
npm run test:smoke # Smoke API (serveur local requis)
```

2. Pyramide de tests

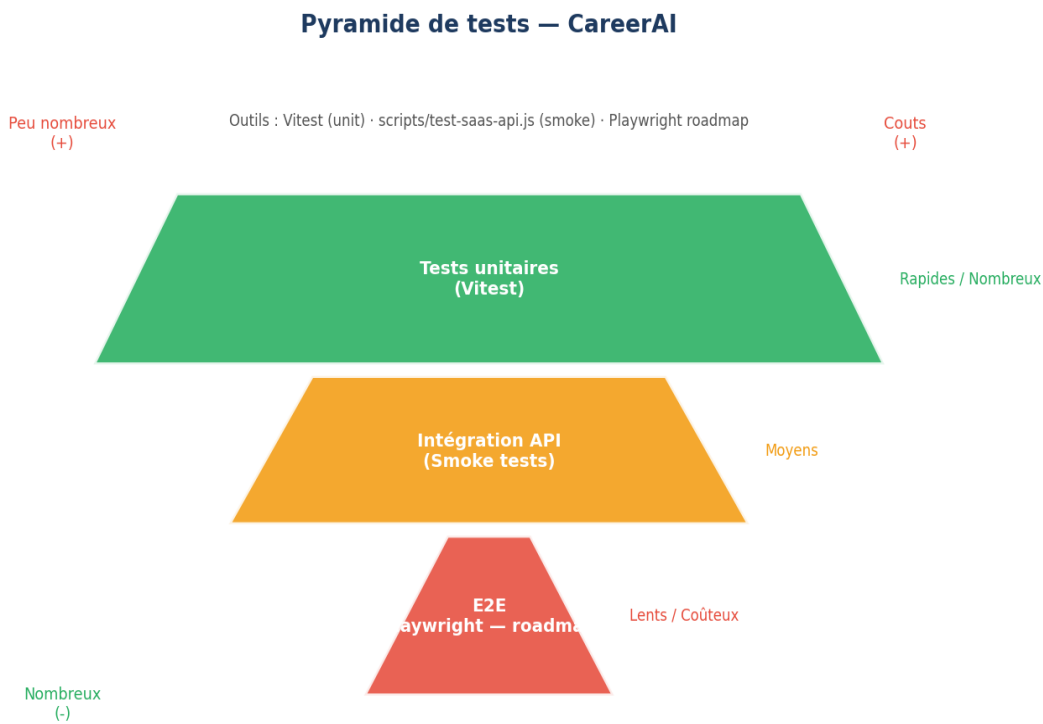


Figure 1 — Pyramide de tests CareerAI

Niveau	Objectif	Outils	Statut
Tests unitaires	Vérifier les règles métier pures (fonctions sans I/O)	Vitest	■ En place
Intégration API	Vérifier les endpoints critiques avec session réelle	scripts/test-saas-api.js	■ Smoke disponible
E2E (Playwright)	Parcours utilisateur complets (inscription, campagne, achat)	Playwright	■ Roadmap V2

3. Cas couverts en V1

Tests unitaires (Vitest)

Cas de test	Type	Périmètre
Détection type de fichier CV (PDF/DOCX/autre)	Nominal + cas limites	Utilitaire détection MIME

Validation simple d'email contact	Nominal + email invalide	Validation entrée recruteur
Calcul de borne de campagne (duration_days, max_per_day)	Nominal + valeurs extrêmes	CampaignService logique

Smoke tests d'intégration

Endpoint testé	Action	Vérification
POST /api/recruiter/job-postings	Création poste recruteur	Statut 200, id renvoyé
POST /api/recruiter/quizzes	Génération quiz	Statut 200, questions présentes
GET /api/recruiter/candidates	Liste candidats	Statut 200, tableau non vide
GET /api/recruiter/rankings	Classements par poste	Statut 200, scores triés

4. Couverture et indicateurs

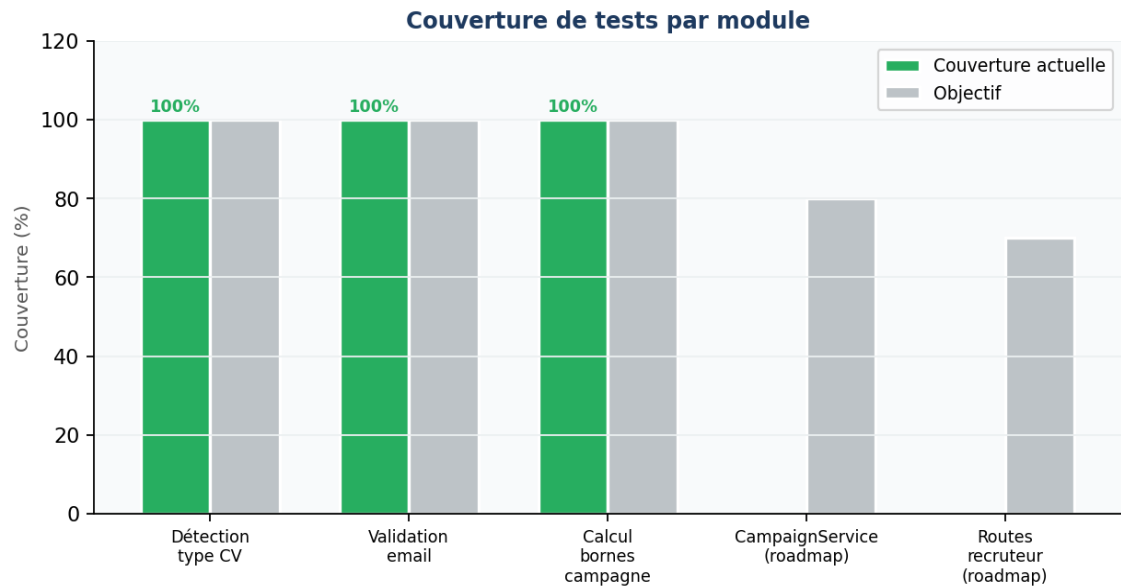


Figure 2 — Couverture actuelle vs objectifs par module

La couverture actuelle est **volontairement partielle** et centrée sur des fonctions pures (sans dépendance Supabase/Resend/Groq). Cette approche pragmatique permet de lancer une base stable sans mock massif des services externes.

Module	Couverture actuelle	Objectif court terme	Priorité
Détection type CV	100%	100%	■ Fait
Validation email	100%	100%	■ Fait
Calcul bornes campagne	100%	100%	■ Fait
CampaignService matching	0%	80%	■ Priorité 1
Validations routes recruteur	0%	70%	■ Priorité 2

Erreurs métier critiques	0%	60%	■ Priorité 3
--------------------------	----	-----	--------------

5. Mécanismes de non-régression

La non-régression est assurée par plusieurs mécanismes complémentaires :

Mécanisme	Description
npm test avant merge	Les tests unitaires Vitest sont exécutés avant tout merge sur develop
CI GitHub Actions	Workflow build-android.yml sur main et develop — build systématique
Smoke tests manuels	Scripts/test-saas-api.js exécutés avant chaque mise en production
Migrations idempotentes	Les migrations SQL peuvent être rejouées sans effet de bord
Playwright (roadmap)	Tests E2E sur parcours auth + campagne pour détecter les régressions UI

6. Roadmap tests

Étape	Action	Délai estimé
V1.1	Tests unitaires CampaignService (matching + formatage)	Court terme
V1.2	Tests intégration API avec base Supabase de test dédiée	Moyen terme
V1.3	Tests non-régression front — pages auth et campagnes	Moyen terme
V2.0	E2E Playwright sur parcours inscription → première campagne	Long terme